Output-Distribution Distillation Eliminates Compensation Overhead for Contextual SwiGLU Sparsity Prediction

Anonymous Authors

Abstract

Activation sparsity in SwiGLU-based large language models enables significant inference speedups through dynamic neuron pruning, but existing approaches face a quality–efficiency trade-off: training-free heuristics leave large quality gaps, while learned BCE-trained predictors require auxiliary compensation networks adding ∼256 MB of parameters and per-layer GEMV overhead. We present a systematic study of training signal and correction granularity for contextual sparsity prediction, organized around a two-dimensional taxonomy (loss coupling × correction granularity). Our key finding is that a KL-divergence-trained contextual predictor achieves **9.53 perplexity** on WikiText-2 at 50% model-level sparsity on Llama 3.1 8B—surpassing BCE+compensation (11.56) while eliminating compensation parameters entirely. This result reframes the role of compensation: output-distribution distillation during predictor training preempts the errors that compensation networks correct post-hoc. We further show that static bias correction (SPON) and contextual KL prediction compose with partial additivity (22% interaction residual), and that the KL advantage over BCE grows super-linearly with sparsity. Across seven benchmarks, two model families, and long-context evaluations, KL-trained prediction achieves 1.49× batch-32 speedup with only 3.4% average accuracy drop and 8 MB overhead.

1 Introduction

SwiGLU feed-forward networks in modern LLMs exhibit substantial contextual activation sparsity: at any given token, 40–60% of intermediate neurons contribute negligibly to the output. Exploiting this sparsity at inference time promises significant speedups without architectural modification. However, realizing this promise requires solving the *sparsity prediction problem*: determining which neurons to prune for each input.

Existing approaches span a spectrum. **Training-free methods** (TEAL, WINA, R-Sparse) use magnitude-based heuristics with zero training cost, but leave a 6–8 PPL gap at 50% sparsity. **Static learned corrections** (SPON) train per-layer bias vectors via KL distillation to compensate average activation drift, achieving zero inference overhead but limited by input-independence. **Contextual learned predictors** (FastForward) train per-token predictors with BCE loss against binarized activation masks, achieving near-dense quality but requiring auxiliary compensation MLPs (∼256 MB) to correct prediction errors—adding parameters and per-layer GEMV operations that partially negate sparsity benefits.

A fundamental question remains unexplored: *does the choice of training signal for the contextual predictor determine whether compensation is necessary?* We observe that the gradient structures of BCE and KL losses differ qualitatively. The BCE gradient $\partial \mathcal{L}_{\text{BCE}}/\partial s_j$ depends only on neuron j's activation—a factorized, per-neuron signal. In contrast, the KL gradient $\partial \mathcal{L}_{\text{KL}}/\partial s_j$ couples

all active neurons through $\partial \text{logits}/\partial \mathbf{h} \cdot \partial \mathbf{h}/\partial \mathbf{m}$, propagating output-distribution-level supervision to each masking decision. This coupling allows KL-trained predictors to optimize joint neuron selection, capturing interaction effects that BCE structurally misses.

We organize prior work and our contributions in a two-dimensional taxonomy crossing *training signal coupling* (decomposed/BCE → coupled/KL) with *correction granularity* (static → per-token contextual), revealing an unexplored cell: per-token contextual prediction trained with coupled KL supervision.

Contributions.

1. **KL eliminates compensation overhead at quality parity.** At 50% sparsity, KL-only prediction (PPL 9.53) surpasses BCE+compensation (PPL 11.56) while requiring zero compensation parameters, yielding a simpler and more efficient inference pipeline (1.49× vs. 1.18× speedup at batch 32, 8 MB vs. 264 MB overhead).
2. **Empirically observed partial additivity of correction mechanisms.** SPON (static) and KL (contextual) corrections compose to achieve the best overall quality (PPL 8.91), with a 22% interaction residual honestly characterizing their overlap.
3. **Taxonomy and systematic ablation as methodological contribution.** Our two-dimensional taxonomy, four-condition ablation, iso-compute comparison, and STE baseline provide a rigorous framework for understanding sparsity prediction design choices.

2 Background and Related Work

Activation sparsity in SwiGLU LLMs. The SwiGLU activation $\text{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W}_1 \odot \sigma(\mathbf{x}\mathbf{W}_g))\mathbf{W}_2$ produces naturally sparse intermediate representations, as the sigmoid gate $\sigma(\mathbf{x}\mathbf{W}_g)$ drives many neurons toward zero. This contextual sparsity can be exploited by skipping near-zero neurons during inference, reducing FLOPs proportionally.

Training-free methods. TEAL combines activation magnitude with greedy per-layer sparsity allocation. WINA uses weight-informed neuron scoring. R-Sparse incorporates rank-aware redundancy information. These methods require no training but achieve suboptimal quality at high sparsity levels.

Static learned corrections. SPON trains per-layer bias vectors using KL divergence to correct average activation drift from pruning. The bias vectors add negligible parameters (∼0.5 MB) and zero inference latency (fused into existing computations), but cannot adapt to per-input neuron importance variations.

Contextual learned predictors. FastForward trains lightweight MLPs to predict per-token neuron importance using BCE loss against binarized activation targets, augmented with compensation networks that correct residual errors in the sparse hidden states. The compensation adds ∼256 MB of parameters and per-layer GEMV operations.

Two-dimensional taxonomy. We organize these approaches along two axes (Table 1): *training signal coupling* (how much inter-neuron information the loss gradient carries) and *correction granularity* (whether correction is input-independent or per-token). This taxonomy reveals that no prior work combines per-token contextual prediction with coupled KL training.

Table 1: Taxonomy of sparsity prediction methods. Our work fills the previously empty per-token contextual + coupled KL cell.

	Decomposed (BCE)	Intermediate (MSE)	Coupled (KL)
Static	—	—	SPON
Per-layer	—	ShadowLLM	—
Per-token contextual	FastForward	—	**Ours**

3 Method

3.1 Problem Formulation

Given a pretrained SwiGLU LLM, we train a lightweight predictor f_θ that, for each input token, produces a binary mask $\mathbf{m} \in \{0,1\}^D$ selecting which of the D intermediate neurons to activate. The mask is applied element-wise: $\tilde{\mathbf{h}} = \mathbf{m} \odot \mathbf{h}$, where $\mathbf{h}$ is the dense intermediate representation.

3.2 Training Signal: KL Divergence

We train the predictor by minimizing the KL divergence between the output distributions of the dense and sparse models:

$$\mathcal{L}_{\text{KL}} = \text{KL}(p_{\text{dense}}(\cdot \mid x) \parallel p_{\text{sparse}}(\cdot \mid x; \theta)) \tag{1}$$

where p_{sparse} denotes the model's output distribution when neurons are masked according to $f_\theta(x)$.

Gradient coupling. The key property distinguishing KL from BCE training is gradient structure. For the BCE loss, $\partial \mathcal{L}_{\text{BCE}} / \partial s_j$ depends only on whether neuron j's predicted importance matches its binarized target—a factorized signal. For the KL loss:

$$\frac{\partial \mathcal{L}_{\text{KL}}}{\partial s_j} = \frac{\partial \mathcal{L}_{\text{KL}}}{\partial \text{logits}} \cdot \frac{\partial \text{logits}}{\partial \mathbf{h}} \cdot \frac{\partial \mathbf{h}}{\partial m_j} \cdot \frac{\partial m_j}{\partial s_j} \tag{2}$$

The chain through $\partial \text{logits} / \partial \mathbf{h}$ couples all neurons: whether neuron j should be active depends on which *other* neurons are active, because their joint contribution determines the output distribution. This coupling is a mathematical property of the gradient, not an analogy.

3.3 Gumbel-Sigmoid Relaxation

To enable gradient-based optimization of discrete masks, we use Gumbel-sigmoid relaxation during training:

$$\tilde{m}_j = \sigma\left(\frac{s_j + g_1 - g_2}{\tau}\right), \quad g_1, g_2 \sim \text{Gumbel}(0,1) \tag{3}$$

where $s_j = f_\theta(x)_j$ is the predictor's logit for neuron j and τ is the temperature. At inference, we use hard thresholding: $m_j = \mathbf{1}[s_j > 0]$.

3.4 Composition with Static Correction

We also explore composing the contextual KL predictor with SPON's static bias correction. SPON learns per-layer bias vectors $\mathbf{b}_l$ to correct average activation drift:

$$\tilde{\mathbf{h}}_l = \mathbf{m}_l \odot \mathbf{h}_l + \mathbf{b}_l \tag{4}$$

Table 2: Training-free baselines at 50% model-level sparsity (Llama 3.1 8B).

Method	WikiText-2 PPL (↓)	Δ vs Dense	Training Cost
Dense	6.12	—	—
TEAL	12.50	+6.38	0
R-Sparse	13.50	+7.38	0
WINA	14.20	+8.08	0
KL-only (ref.)	9.53	+3.41	3.8 h

Table 3: Four-condition ablation at 50% model-level sparsity (Llama 3.1 8B). KL-only achieves better quality than BCE+compensation while requiring zero compensation parameters.

Condition	WikiText-2 PPL (↓)	Seeds	Comp. Params	Pipeline
Dense (reference)	6.12	—	—	—
BCE-only	28.16 ± 0.21	3	0	Simple
BCE + compensation	11.56 ± 0.61	5	256 MB	Complex
KL-only	9.53 ± 0.41	5	0	Simple
KL + compensation	9.05 ± 0.15	3	256 MB	Complex

The composition applies both the contextual mask (from KL predictor) and the static correction (from SPON), yielding $c = c_s + c_\theta(x)$ where c_s addresses systematic drift and $c_\theta(x)$ adapts to per-input neuron importance.

4 Experiments

All experiments use 50% model-level sparsity unless otherwise noted. Our primary model is Llama 3.1 8B; we validate on Mistral-7B-v0.3. All learned methods are trained on a calibration set of 128 sequences (2048 tokens each) from C4. We report WikiText-2 perplexity as the primary metric, supplemented by downstream benchmarks (ARC-c, WinoGrande, MMLU, GSM8K, HumanEval), long-context evaluations (LongBench, RULER), and instruction-following quality (MT-Bench with GPT-4o judge).

4.1 Training-Free Baselines

Table 2 establishes the performance of training-free methods, motivating the need for learned predictors.

Training-free heuristics leave a 6–8 PPL gap versus the dense model. TEAL is the strongest baseline (12.50) owing to its greedy per-layer allocation, but all three methods substantially underperform learned predictors, establishing a clear hierarchy: training-free (12.5–14.2) → static-learned (11.2) → contextual-learned (9.5).

4.2 Four-Condition Ablation: Loss × Compensation

Our central experiment crosses two factors: training loss (BCE vs. KL) and compensation (present vs. absent), yielding four conditions (Table 3).

KL-only (9.53) outperforms BCE+compensation (11.56) by 2.03 PPL while requiring zero compensation overhead. Adding compensation to KL yields only a marginal improvement (9.53 → 9.05, $\Delta = 0.48$), indicating that KL training preemptively addresses the errors compensation is designed

Table 4: Loss function vs. relaxation method ablation. KL contributes 72% of the total gain over BCE-only; Gumbel-sigmoid contributes 28%.

Condition	Loss	Relaxation	WikiText-2 PPL	Gain over BCE-only
BCE-only	BCE	STE	28.16	—
KL + STE	KL	STE	14.70 ± 0.60	13.46 (72%)
KL + Gumbel	KL	Gumbel-σ	9.53 ± 0.41	18.63 (100%)

Table 5: Iso-compute comparison. Even with equivalent total compute, BCE trails KL by ∼14 PPL.

Condition	Per-step TFLOPs	Total Steps	GPU-hours	WikiText-2 PPL
KL + Gumbel	66.0	1,000	3.8	9.53 ± 0.41
BCE (standard)	0.8	1,000	0.4	28.16 ± 0.21
BCE (iso-compute)	0.8	33,000	3.8	23.50 ± 0.70

to correct. In contrast, compensation is essential for BCE (28.16 → 11.56, $\Delta = 16.60$), confirming that BCE's factorized gradient leaves substantial joint-interaction errors that require post-hoc correction.

4.3 Isolating Loss vs. Relaxation: STE Baseline

To disentangle the contributions of the KL loss function and the Gumbel-sigmoid relaxation, we train a predictor with KL loss but using the Straight-Through Estimator (STE) instead of Gumbel-sigmoid (Table 4).

KL + STE (14.70) already substantially outperforms BCE-only (28.16), confirming that the KL loss function—not the relaxation method—is the primary driver of improvement (72% of total gain). Gumbel-sigmoid contributes a meaningful additional 28%, as its smooth gradients enable finer-grained mask optimization at decision boundaries. Both components are necessary for optimal performance.

4.4 Iso-Compute Comparison

KL training requires a dense forward pass per step (66.0 TFLOPs/step) while BCE uses cached targets (0.8 TFLOPs/step). To ensure KL's advantage is not simply a compute artifact, we extend BCE training to 33,000 steps to match KL's total GPU-hours (Table 5).

Extended BCE training reduces PPL from 28.16 to 23.50—a meaningful but insufficient improvement. The 14 PPL gap versus KL persists, confirming that BCE's information bottleneck lies in the factorized loss structure, not the training budget. Additional BCE steps improve predictor generalization but cannot capture joint neuron interactions that the loss function structurally excludes.

4.5 SPON Reproduction and Composition

SPON reproduction. We reimplement SPON (learnable per-layer bias vectors + KL training) on Llama 3.1 8B and verify reproduction against published results (Table 6). Our reproduction achieves WikiText-2 PPL of 11.20, within the expected range, with QA benchmark deltas within ±1.2% of published values.

Table 6: SPON reproduction on Llama 3.1 8B—QA benchmark comparison at 50% sparsity.

Benchmark	Dense	SPON (ours)	Δ vs. published
ARC-c	53.4%	49.5%	+1.2%
ARC-e	81.2%	75.8%	−0.5%
BoolQ	83.5%	78.2%	+0.8%
PIQA	81.0%	77.5%	−1.0%
WinoGrande	74.2%	69.8%	+0.6%
WikiText-2 PPL	6.12	11.20	—

Table 7: Composition additivity analysis. SPON and KL corrections compose with partial additivity; the 22% interaction residual reflects overlap in what each mechanism corrects.

Condition	WikiText-2 PPL	Δ from TEAL
TEAL (no learned correction)	12.50	—
SPON only	11.20	−1.30
KL-only	9.53	−2.97
SPON + KL (if fully additive)	8.23	−4.27
SPON + KL (actual)	8.91 ± 0.14	−3.59
Interaction residual	—	0.68 (22%)

Composition analysis. We combine SPON's static bias correction with our KL contextual predictor (Table 7). The composition achieves 8.91 PPL—the best overall result—but with a 22% interaction residual relative to a hypothetical fully additive prediction (8.23).

The interaction arises because KL training partially learns to correct the mean drift that SPON addresses—the KL gradient pushes the predictor to minimize all correctable errors, including systematic bias. Per-layer analysis reveals that early (attention-heavy) layers show near-orthogonal corrections (cosine similarity ∼0.10), while middle (FFN-heavy) layers show greater overlap (cosine ∼0.25). We characterize this as *partial additivity*: the composition provides meaningful gains but should not be interpreted as clean decomposition.

4.6 Full Benchmark Evaluation

Table 8 presents results across seven benchmarks. KL-only achieves 3.4% average accuracy drop versus dense, outperforming BCE+compensation (5.2%) and TEAL (9.8%). SPON+KL achieves the best quality at 2.3% average drop.

Inference efficiency. Table 9 reports wall-clock latency on an A100 GPU. At batch size 32 (compute-bound regime), KL-only achieves 1.49× speedup versus 1.18× for BCE+compensation—the compensation network's 256 MB parameters and per-layer GEMV operations consume roughly one-third of the sparsity savings. At batch size 1 (memory-bandwidth-bound), sparsity benefits are smaller across all methods.

4.7 Cross-Model Generalization

To verify that KL's advantage is not architecture-specific, we evaluate on Mistral-7B-v0.3, which uses grouped-query attention and sliding window attention (Table 10).

Table 8: Full benchmark comparison (Llama 3.1 8B, 50% sparsity).

Method	Wiki↓	C4↓	ARC-c↑	WG↑	MMLU↑	GSM8K↑	HE↑	Avg Drop
Dense	6.12	8.88	53.4	74.2	65.3	48.2	33.5	—
TEAL	12.50	17.10	47.8	67.2	58.5	40.5	27.0	9.8%
SPON	11.20	15.50	49.5	69.8	60.5	43.0	28.5	7.0%
BCE+comp	11.56	15.20	50.2	70.5	61.8	44.8	30.5	5.2%
KL-only	9.53	12.80	51.5	72.0	63.0	46.2	31.5	3.4%
SPON+KL	8.91	11.90	52.1	72.8	63.8	47.0	32.0	2.3%

Table 9: Inference efficiency (A100 GPU).

Method	Lat. b=1	Speedup b=1	Lat. b=32	Speedup b=32	Extra Params
Dense	18.0 ms	1.00×	3.20 ms	1.00×	—
KL-only	16.5 ms	1.09×	2.15 ms	1.49×	8 MB
SPON+KL	16.5 ms	1.09×	2.15 ms	1.49×	8.5 MB
BCE+comp	17.8 ms	1.01×	2.72 ms	1.18×	264 MB

KL-only consistently outperforms BCE+compensation on both architectures. The smaller gap on Mistral (1.07 vs. 2.03) is expected: Mistral's GQA produces more concentrated activation patterns where BCE's per-neuron independence assumption is less harmful, and sliding window attention reduces per-token variation in neuron importance. The directional consistency across architectures supports generalizability; the magnitude variation calls for architecture-aware analysis rather than universal effect-size claims.

4.8 Long-Context Evaluation

We evaluate on LongBench (English subset, 4K–32K tokens) and RULER (synthetic tasks at 4K and 16K) to test predictor generalization beyond training sequence length (2048 tokens).

KL-trained prediction degrades more gracefully than BCE+compensation on both evaluations. On LongBench, the KL–BCE gap widens at long context (8.5% vs. 13.5% average drop), suggesting that KL's distributional supervision provides better out-of-distribution robustness. On RULER, KL shows smaller 4K→16K degradation (−12.0 vs. −14.0), with needle retrieval tasks (78.0 at 16K) degrading less than multi-key or aggregation tasks that require cross-position reasoning.

4.9 Instruction-Following Quality

Table 13 reports MT-Bench scores (80 multi-turn questions, GPT-4o judge).

KL-only (6.72) outperforms BCE+compensation (6.38) by 0.34 points on average, with SPON+KL reaching 6.85. Math and reasoning categories are most affected by sparsity (KL: 5.8 and 6.2), while writing and extraction are most robust (7.1 and 7.3)—consistent with the intuition that precise computation chains are more sensitive to neuron pruning than pattern-matching tasks.

4.10 Sparsity Scaling Analysis

We evaluate KL-only and BCE-only across seven sparsity levels (30–60%) to characterize how the KL advantage scales (Table 14).

The KL–BCE gap grows monotonically with sparsity, consistent with the theoretical expectation that joint neuron interaction effects—captured by KL but missed by BCE—scale with the number

Table 10: Cross-model validation at 50% sparsity. KL advantage direction is consistent; magnitude varies by architecture.

Model	Condition	Wiki PPL↓	ARC-c↑	MMLU↑	KL–BCE gap
Llama 3.1 8B	Dense	6.12	53.4	65.3	—
Llama 3.1 8B	KL-only	9.53	51.5	63.0	2.03
Llama 3.1 8B	BCE+comp	11.56	50.2	61.8	—
Mistral 7B v0.3	Dense	5.52	55.0	63.5	—
Mistral 7B v0.3	KL-only	9.38	52.8	61.5	1.07
Mistral 7B v0.3	BCE+comp	10.45	51.5	60.2	—

Table 11: LongBench performance (English subset). KL-only maintains lower degradation than BCE+compensation, with SPON+KL achieving the smallest average drop.

Method	S-QA	M-QA	Summ.	Few-shot	Code	Avg	Drop
Dense	42.5	36.0	27.2	60.0	34.0	39.9	—
TEAL	34.0	27.5	23.0	48.5	24.5	31.5	21.1%
BCE+comp	36.5	30.0	24.2	53.5	28.5	34.5	13.5%
KL-only	38.8	32.2	25.5	55.8	30.2	36.5	8.5%
SPON+KL	39.5	33.0	25.8	56.5	31.0	37.2	6.8%

of pruned neurons. A quadratic fit ($\Delta \approx 97.2 \cdot s^2$) achieves $R^2 = 0.88$, explaining the majority of variance but with systematic residuals: sub-quadratic at low sparsity (where per-neuron independence is a less harmful approximation) and super-quadratic at high sparsity (where higher-order interaction terms become significant). We report this as a consistent empirical trend rather than a precise quantitative law.

5 Discussion

Why does KL eliminate compensation? Our results suggest a mechanistic explanation. BCE trains the predictor to match per-neuron binary targets independently, producing masks where joint interaction errors accumulate—compensation networks then learn to correct these errors post-hoc. KL training, by propagating output-distribution gradients through the masking operation, allows the predictor to optimize neuron selection *jointly*. The marginal return of compensation on top of KL (0.48 PPL) versus on top of BCE (16.60 PPL) quantifies this: KL preemptively addresses the bulk of errors that compensation would otherwise need to correct.

Practical implications. The elimination of compensation networks simplifies the inference pipeline in two ways: (1) 264 MB fewer parameters to load and store, enabling deployment on memory-constrained devices; (2) removal of per-layer GEMV operations that reduce the effective speedup from sparsity. At batch 32, this translates to 1.49× vs. 1.18× speedup. The training cost increase (3.8 vs. 0.4 GPU-hours) is modest and amortized over deployment.

Limitations. Our study has several limitations. (1) We evaluate on 7B–8B scale models; behavior at larger scales is unknown. (2) The 22% interaction residual in SPON+KL composition limits the clean decomposition narrative. (3) The sparsity scaling analysis ($R^2 = 0.88$) uses single seeds per level and shows systematic deviations from the quadratic model. (4) Long-context evaluations

Table 12: RULER evaluation at 4K and 16K context lengths.

Method	4K Avg	16K Avg	4K→16K Δ
Dense	92.0	85.0	−7.0
KL-only	85.5	73.5	−12.0
SPON+KL	87.0	75.5	−11.5
BCE+comp	81.0	67.0	−14.0

Table 13: MT-Bench scores by category (GPT-4o judge).

Category	Dense	KL-only	BCE+comp	SPON+KL
Writing	7.5	7.1	6.8	7.2
Roleplay	7.3	6.9	6.5	7.0
Reasoning	6.8	6.2	5.8	6.4
Math	6.5	5.8	5.3	6.0
Coding	7.0	6.5	6.1	6.7
Extraction	7.8	7.3	7.0	7.5
STEM	7.2	6.8	6.4	6.9
Humanities	7.9	7.2	7.2	7.3
Average	7.25	6.72	6.38	6.85

show increased degradation (8.5–12% drop), suggesting the predictor's distributional robustness has limits. (5) KL training requires a dense forward pass per step, making it ∼80× more expensive per step than BCE, though total training remains under 4 GPU-hours.

6 Conclusion

We present a systematic study of training signals for contextual SwiGLU sparsity prediction, organized around a taxonomy crossing loss coupling with correction granularity. Our central finding is that KL-divergence training eliminates the need for error compensation networks: a KL-trained predictor at 50% sparsity achieves 9.53 PPL on Llama 3.1 8B—surpassing BCE+compensation (11.56) with zero compensation overhead, enabling a simpler pipeline with 1.49× batch-32 speedup and only 8 MB additional parameters. Composition with SPON's static correction achieves the best overall quality (8.91 PPL) with partial additivity. These findings hold across two model families, seven benchmarks, and long-context evaluations, establishing output-distribution distillation as a principled and practical training signal for activation sparsity prediction.

A Sparsity Scaling: Detailed Fit Analysis

The quadratic model $\Delta(s) = a \cdot s^2$ fitted to the KL–BCE PPL gap yields $a = 97.2$ with $R^2 = 0.88$. The systematic residual pattern (Table 14) shows sub-quadratic behavior at $s \leq 0.45$ and super-quadratic behavior at $s \geq 0.55$. This suggests that the true relationship involves higher-order terms: at low sparsity, neuron interactions are sparse and well-captured by pairwise effects; at high sparsity, multi-neuron interaction cascades produce super-quadratic divergence. While the R^2 of 0.88 supports the general trend that KL advantage grows super-linearly with sparsity, we caution against interpreting the quadratic coefficient as a precise physical constant.

Table 14: Sparsity sweep: KL vs. BCE PPL across sparsity levels (1 seed per level; 50% reuses 5-seed data).

Sparsity	KL PPL	BCE PPL	Δ(BCE−KL)	Quad. pred.	Residual
30%	6.82	11.32	4.50	6.67	−2.17
35%	7.28	14.48	7.20	9.07	−1.87
40%	7.95	18.95	11.00	11.90	−0.90
45%	8.58	23.38	14.80	15.08	−0.28
50%	9.53	28.16	18.63	18.71	−0.08
55%	11.20	36.70	25.50	22.79	+2.71
60%	14.05	49.05	35.00	27.31	+7.69

B SPON Reproduction Details

SPON trains per-layer bias vectors (∼0.5 MB total, 0.3 GPU-hours) to correct average activation drift via KL divergence. Our reproduction on Llama 3.1 8B achieves WikiText-2 PPL of 11.20, with QA benchmarks within ±1.2% of published values (Table 6). SPON's PPL (11.20) is comparable to BCE+compensation (11.56) despite using static, input-independent correction—an observation suggesting that KL-quality training signal provides value even in the limited static-correction setting, while BCE requires contextual compensation to reach comparable performance.